STAR RX72N - rx_motor Library

1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Directory Hierarchy

1.1 Directories

Chapter 2

File Index

2.1 File List

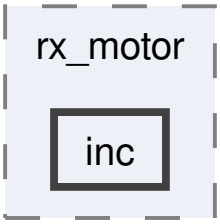
Here is a list of all files with brief descriptions:

Chapter 3

Directory Documentation

3.1 libs/rx_motor/inc Directory Reference

Directory dependency graph for inc:

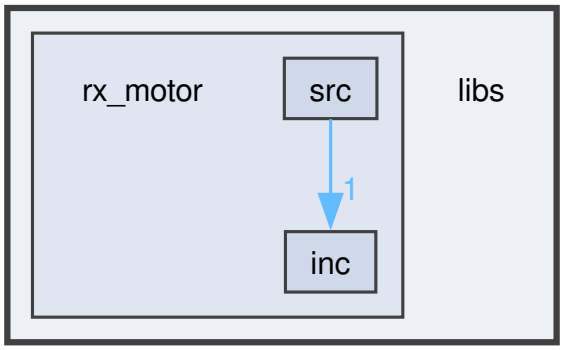


Files

- file rx_motor.h
  Brushed DC Motor Control API using RX72N GPTW PWM with H-Bridge Driver Support.

3.2 libs Directory Reference

Directory dependency graph for libs:

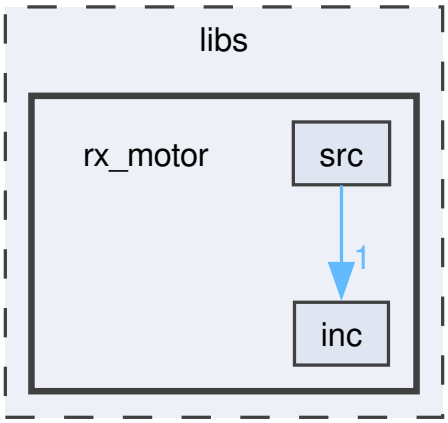


Directories

- directory rx_motor

3.3 libs/rx_motor Directory Reference

Directory dependency graph for rx_motor:

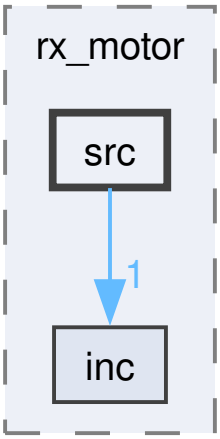


Directories

- directory inc
- directory src

3.4 libs/rx_motor/src Directory Reference

Directory dependency graph for src:



Files

- file rx_motor.c

  Brushed DC Motor Control Implementation for RX72N GPTW PWM with DRV8263H H-Bridge.

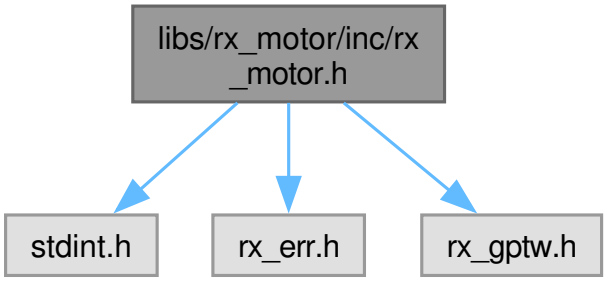
Chapter 4

File Documentation

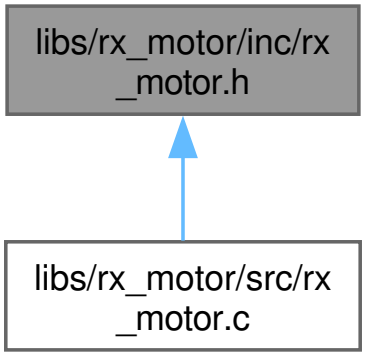
4.1 libs/rx_motor/inc/rx_motor.h File Reference

Brushed DC Motor Control API using RX72N GPTW PWM with H-Bridge Driver Support.

#include <stdint.h>
#include "rx_err.h"
#include "rx_gptw.h"
Include dependency graph for rx_motor.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct rx_motor_config_t
  Motor controller configuration structure for GPTW PWM initialization.
- struct rx_motor_handle_t
  Motor controller handle structure maintaining runtime state and configuration.

Functions

- rx_err_t rx_motor_init (rx_motor_handle_t *handle, const rx_motor_config_t *config)
  Initialize motor controller with GPTW PWM.
- rx_err_t rx_motor_deinit (rx_motor_handle_t *handle)
  Deinitialize motor controller and release resources.
- rx_err_t rx_motor_set_duty (rx_motor_handle_t *handle, float duty)
  Set motor duty cycle.
- rx_err_t rx_motor_stop (rx_motor_handle_t *handle, bool brake)
  Stop motor with active brake or coast.
- rx_err_t rx_motor_get_duty (const rx_motor_handle_t *handle, float *out_duty)
  Get current motor duty cycle.
- rx_err_t rx_motor_emergency_stop (rx_motor_handle_t *handle)
  Emergency stop - disable GPTW outputs immediately.

4.1.1 Detailed Description

Brushed DC Motor Control API using RX72N GPTW PWM with H-Bridge Driver Support.

4.1.1.1 Overview

Production-quality API for controlling brushed DC gearmotors via H-bridge drivers using the RX72N General PWM Timer (GPTW) peripheral. Provides bidirectional control with hardware-generated PWM, automatic dead-time insertion for shoot-through prevention, and fail-safe emergency stop capability.

Application: STAR robot platform - 4x 6V brushed DC gearmotors (210 RPM, 341 PPR encoders) H-Bridge Driver: DRV8263H dual H-bridge with integrated current sensing PWM Peripheral: RX72N GPTW (General PWM Timer) - 32-bit resolution

Key Features:

- Bidirectional control: -100% (full reverse) to +100% (full forward)
- Hardware dead-time insertion: Prevents H-bridge shoot-through (configurable, typically 1us)
- Configurable PWM frequency: Typically 20 kHz (above audible range, below switching losses)
- Brake and coast modes: Active braking (short-circuit) or free-wheeling coast
- Hardware PWM generation: Zero CPU overhead during steady-state operation
- 32-bit timer resolution: GPTW provides higher precision than MTU3's 16-bit
- Emergency stop: Hardware-level output disable for fail-safe operation

4.1.1.2 H-Bridge Control Modes (DRV8263H-Q1 IN/IN Mode)

4.1.1.2.1 IN/IN PWM (Implemented)

Each half-bridge is controlled independently. The active half-bridge carries the PWM signal while the other is held LOW. Both LOW = active brake, both HIGH = coast.

Forward (duty > 0):

- Output A (IN2): Held low (0%)
- Output B (IN1): PWM active (|duty| cycle)
- Motor current: High-side B -> Motor -> Low-side A (forward rotation)

Reverse (duty < 0):

- Output A (IN2): PWM active (|duty| cycle)
- Output B (IN1): Held low (0%)
- Motor current: High-side A -> Motor -> Low-side B (reverse rotation)

Active Brake (duty = 0, or rx_motor_stop with brake=true):

- Output A (IN2): Held low (0%)
- Output B (IN1): Held low (0%)
- Motor: Shorted through low-side FETs (active braking)

Coast (rx_motor_stop with brake=false):

- Output A (IN2): Held high (100%)
- Output B (IN1): Held high (100%)
- Motor: Free-wheeling (high impedance, back-EMF decay through body diodes)

H-Bridge Truth Table (DRV8263H-Q1 IN/IN Mode):

IN2 (Output A)	IN1 (Output B)	Motor State	Current Direction
LOW	PWM	Forward	B -> Motor -> A
PWM	LOW	Reverse	A -> Motor -> B
LOW	LOW	Active Brake	Short circuit (low-side)
HIGH	HIGH	Coast	None (free-wheel, Hi-Z)

4.1.1.3 Dead-Time Protection

Dead-time prevents simultaneous conduction of high-side and low-side FETs (shoot-through), which would create a short circuit from power supply to ground, potentially destroying the H-bridge.

How It Works:

- When switching from HIGH to LOW, insert delay before complementary switch turns ON
- GPTW hardware automatically inserts dead-time (configured via dead_time_ns parameter)
- Typical value: 1000 ns (1 us) - accounts for FET turn-off time (~500-800 ns)

Shoot-Through Scenario (WITHOUT Dead-Time):

```
Time ->
High-side FET: ############.....  (turning off)
Low-side FET:  ............########  (turning on)
                        ^
               OVERLAP = SHOOT-THROUGH!
               (Both FETs ON simultaneously)
```

Protected with Dead-Time:

```
High-side FET: ############.............
Dead-time:     ............|---|.........
Low-side FET:  ..................########
                          ^
                 SAFE GAP (1 us)
```

4.1.1.4 PWM Frequency Selection

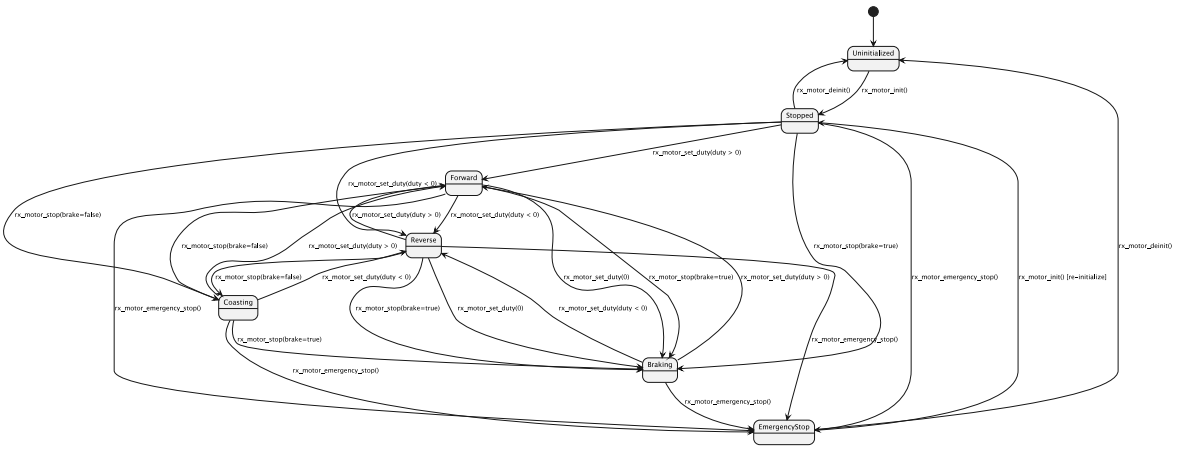
Trade-offs:

Frequency	Advantages	Disadvantages
< 1 kHz	Low switching losses	Audible noise, torque ripple
10-25 kHz (typical)	Inaudible, good efficiency	Moderate EMI
> 50 kHz	Low torque ripple	High switching losses, EMI

STAR Configuration: 20 kHz

- Above human hearing range (20 Hz - 20 kHz) -> silent operation
- Low enough for efficient FET switching (~50 ns rise/fall times)
- Manageable EMI with proper PCB layout
- Standard for brushed DC motor PWM

4.1.1.5 Motor State Machine



State Descriptions:

- Uninitialized: Handle structure exists, GPTW not configured
- Stopped: Motor stopped (coast), GPTW configured and ready
- Forward: Motor driving forward (IN1/output_b active PWM, IN2/output_a LOW)
- Reverse: Motor driving reverse (IN2/output_a active PWM, IN1/output_b LOW)
- Coasting: Motor coasting (both outputs HIGH, Hi-Z free-wheeling)
- Braking: Active braking (both outputs LOW, short-circuit brake)
- EmergencyStop: Hardware outputs disabled, requires re-init

4.1.1.6 Hardware Requirements

Component	Specification	Notes
MCU	Renesas RX72N	GPTW peripheral required
PWM Timer	GPTW (General PWM Timer)	32-bit resolution, 4 channels (GPTW0-3)
H-Bridge	DRV8263H or compatible	Dual H-bridge, 3.5A continuous, 4.5A peak
Motor	6V brushed DC gearmotor	210 RPM no-load, 341 PPR Hall encoder
Power Supply	6-12V DC	Motor voltage range
GPIO Pins	2 per motor (GTIOC A/B)	PWM output pins from GPTW
Dead-Time	Hardware-generated	GPTW feature, configured via API

4.1.1.6.1 GPTW Channel Allocation (STAR Platform)

Motor	GPTW Channel	Output A Pin	Output B Pin	Notes
Motor 0	GPTW0	GTIOC0A (P23)	GTIOC0B (P17)	Front-left

Motor	GPTW Channel	Output A Pin	Output B Pin	Notes
Motor 1	GPTW1	GTIOC1A (P22)	GTIOC1B (PC3)	Front-right
Motor 2	GPTW2	GTIOC2A (PE3)	GTIOC2B (P8.6)	Rear-left
Motor 3	GPTW3	GTIOC3A (PE7)	GTIOC3B (PC6)	Rear-right

4.1.1.7 Performance Characteristics

4.1.1.7.1 Computational Cost (RX72N @ 240 MHz)

Operation	Cycles	Time	Notes
rx_motor_init()	~500	2.1 us	One-time GPTW configuration
rx_motor_set_duty()	~200	833 ns	Update PWM duty cycle
rx_motor_stop()	~150	625 ns	Stop/brake command
rx_motor_get_duty()	~50	208 ns	Read current duty
rx_motor_emergency_stop()	~100	417 ns	Hardware disable

Control Loop Integration:

- Motor control loop: 250 Hz (4 ms period) = 960,000 cycles available
- rx_motor_set_duty(): 200 cycles (0.02% of budget)
- Leaves 959,800 cycles for PID, encoder reading, communication

4.1.1.7.2 Memory Footprint

Component	Size	Count	Total
rx_motor_handle_t	32 bytes	4 motors	128 bytes
Code (Flash)	~2 KB	-	Shared across all motors

4.1.1.8 Usage Examples

See inline function documentation for comprehensive examples including:

- Basic motor initialization and control
- Closed-loop velocity control with PID
- Emergency stop handling
- Brake vs. coast modes
- Error handling patterns

NASA Power of 10 Compliance:

- Rule 1 [YES]: No goto, setjmp/longjmp, recursion
- Rule 2 [YES]: All loops bounded (hardware register access, no unbounded loops)
- Rule 3 [YES]: No dynamic allocation (stack-based handle structure)
- Rule 4 [YES]: All functions < 60 lines
- Rule 5 [YES]: Minimum 2 validations per function (NULL checks, initialization state)
- Rule 6 [YES]: Data at smallest scope
- Rule 7 [YES]: All return values validated (rx_err_t checked at call sites)
- Rule 8 [YES]: No preprocessor macros except include guards
- Rule 9 [YES]: Single-level pointer dereferencing
- Rule 10 [YES]: Compiles with -Wall -Wextra -Werror

SOLID Principles:

- S (Single Responsibility): Module does ONLY motor PWM control - no PID, encoders, or high-level logic
- O (Open/Closed): Configurable via rx_motor_config_t without modifying code
- L (Liskov Substitution): All functions return rx_err_t with consistent semantics
- I (Interface Segregation): Minimal API (6 functions), each with single clear purpose
- D (Dependency Inversion): Depends on abstract rx_gptw.h interface, not direct register access

Module Dependencies:

```
rx_motor.h
  +--> rx_err.h (error code definitions)
  +--> rx_gptw.h (GPTW PWM peripheral abstraction)
  |    +--> rx72n_gptw_regs.h (GPTW register definitions)
  +--> stdbool.h, stdint.h (standard C types)

Used by:
  +--> Motor control tasks (velocity/position closed-loop)
  +--> lib/rx_encoder/ (encoder feedback for motor control)
  +--> Application-level robot control
```

Author

Locked, Inc.

Date

2026-01-27

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT